

UML Diagrams of the Remote File Folder Client Project (a simplified Google Drive analogue)

Yana Utochkina

Stage: Architecture and Behavior Design
September 12, 2026

Abstract

This document presents a set of UML diagrams for the **Remote File Folder Client** project — an application for working with a remote file folder (a simplified analogue of Google Drive) with web and desktop clients, a Go back-end, PostgreSQL, and file synchronization based on diff manifests. The diagrams are ordered according to the design lifecycle: from requirements analysis and high-level architecture to the detailed design of individual subsystems. All diagrams were built using Mermaid notation.

Contents

1 Requirements Analysis	2
2 Architectural Design (High Level)	2
2.1 Use Case Diagram	2
2.2 Class Diagram (General Structure)	4
2.3 Deployment Diagram (General Architecture)	6
3 Detailed Design	7
3.1 VOPC Class Diagram for the “Synchronize local folder with remote” use case	8
3.2 Object Diagram (Conflict Snapshot)	10
3.3 Interaction Diagrams (Sequence & Communication Diagrams) . .	12
3.4 State Diagram	20
3.5 Component Diagram	22

3.6 Package Diagram	23
3.7 Activity Diagram	24
4 Conclusions	25

1. Requirements Analysis

The system gives an authenticated user access to a personal remote folder (a “virtual drive”), where they can view the list of files along with their attributes (name, creation date, modification date, uploader, last editor), upload, download, and delete files, as well as synchronize a chosen local directory with the remote one.

Key functional requirements:

- user authentication on the remote server;
- viewing file attributes with the ability to show/hide columns (the file name column cannot be hidden);
- viewing the content of specific file types: `.java` – as text, `.png` – as an image;
- sorting files by the username of the last editor (ascending / descending);
- filtering files: all files / only `.cs` / only `.jpg`;
- synchronizing the local folder with the remote one based on manifest comparison (name, size, mtime, sha256) and handling editing conflicts.

Architecturally, the system consists of a React client (web), a Swift client (macOS, with full background synchronization via `FSEvents`), a Go API server (handler / service / repository layers), PostgreSQL for metadata, and an abstract file storage (local disk or S3-compatible storage), deployed on a home Ubuntu server and published via Cloudflare Tunnel.

Based on these requirements, the following sections build a use case diagram as well as architectural and detailed diagrams describing the structure and behavior of the system.

2. Architectural Design (High Level)

This stage defines the overall system architecture: who its users are and which scenarios it supports (use case diagram), the general data structure (high-level class diagram), and the physical deployment topology (deployment diagram).

2.1 Use Case Diagram

The diagram (fig. 1) shows a single *User* actor who works with the system through a web or desktop client, and the main use cases corresponding to the functional requirements from section 1. Some of the file operations *include* («include») the “*Authenticate*” use case, since access to the workspace is only possible after a successful login. Viewing file content is *extended* («extend») by two alternative display scenarios (`.java` / `.png`), and synchronization is extended by a conflict-resolution scenario that is activated only when discrepancies exist.

Notation note: Mermaid has no built-in use case diagram type, so the actor,

use cases, and system boundary were built using flowchart (actor - rounded rectangle, use cases - “stadium” shape, system boundary - subgraph), with explicit stereotype labels «include»/«extend» on the relations.

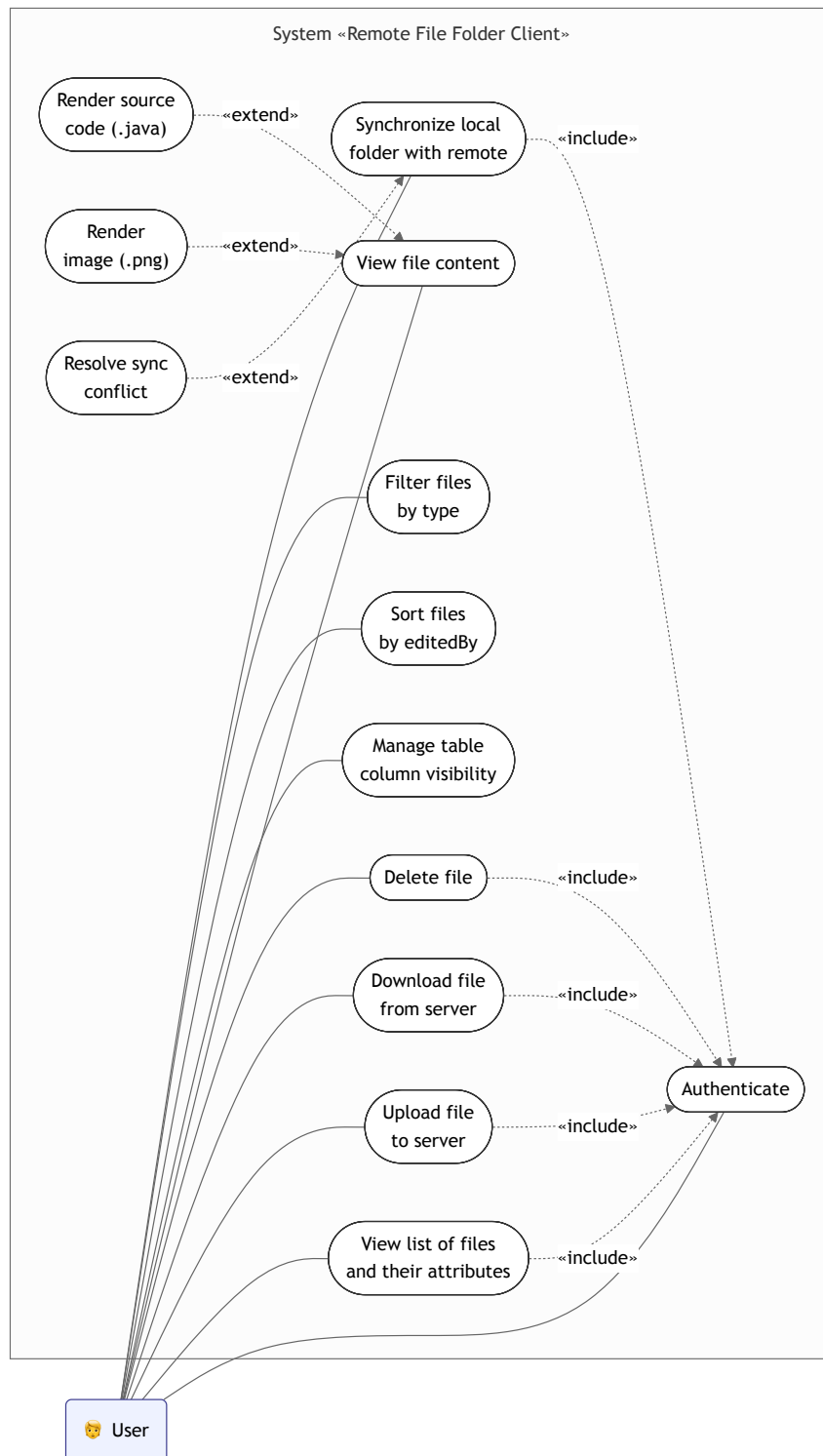


Figure 1: Use case diagram of the Remote File Folder Client system

2.2 Class Diagram (General Structure)

Fig. 2 displays the general data structure and the main relationships between the domain classes of the system, without detailing the methods of individual scenarios: the User and File entities, the service layer (AuthService, FileService, SyncDiffService), the data access layer (FileRepository), and the FileStorage abstraction, implemented by local disk storage and S3-compatible storage – following the “storage behind an interface” principle laid out in the requirements (sections 5.2, 5.5).

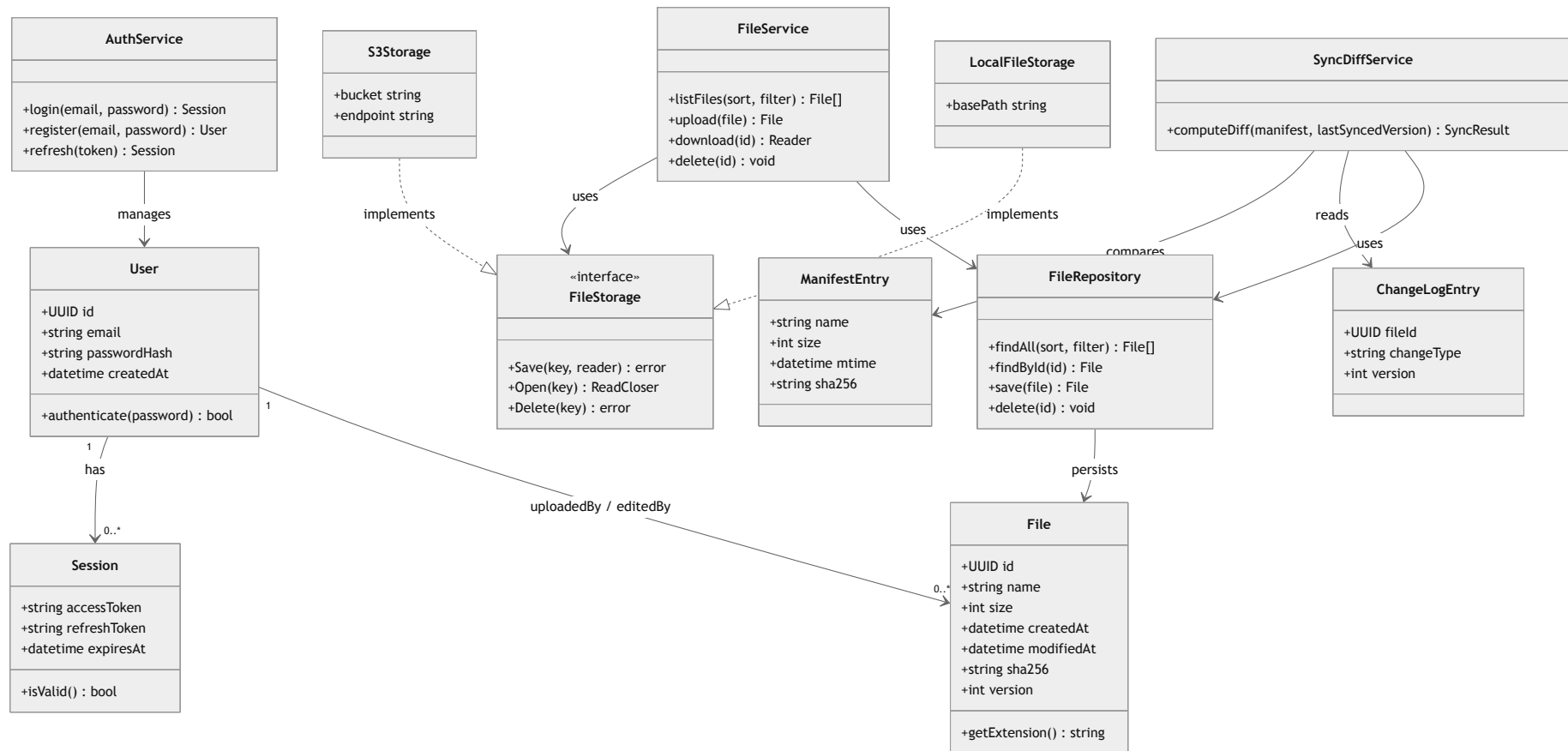


Figure 2: High-level class diagram (domain model)

2.3 Deployment Diagram (General Architecture)

Fig. 3 displays the physical topology of the system: client devices (browser, macOS), the Cloudflare edge (TLS termination, public DNS), and a home Ubuntu server that, via Docker Compose, runs three containers – `cloudflared` (out-bound tunnel without opening ports), `api` (the Go binary), and `db` (PostgreSQL) – together with a persistent `pgdata` volume and a bind-mounted directory for file storage.

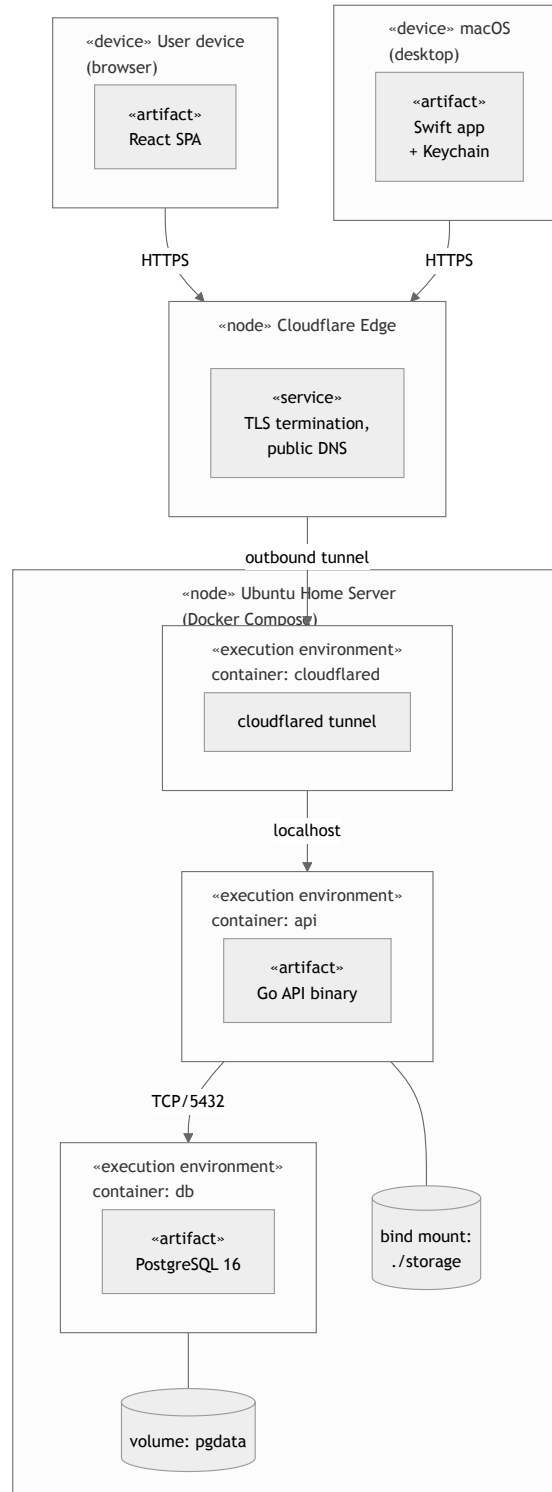


Figure 3: Deployment diagram

3. Detailed Design

Detailed design reveals the internal structure and behavior of individual sub-systems: the classes participating in the implementation of a specific use case (VOPC diagram), a concrete runtime snapshot of objects during a conflict (ob-

ject diagram), the interaction of objects over time – shown both in a time-ordered (sequence) and a structural (communication) notation –, the behavior of a file depending on its synchronization state (state diagram), the structure of software components (component diagram), the modular organization of the code (package diagram), and the business-process logic of synchronization (activity diagram).

3.1 VOPC Class Diagram for the “Synchronize local folder with remote” use case

According to the assignment variant, the use case chosen for the VOPC diagram (*View Of Participating Classes*) is “**Synchronize local folder with remote**” – the most complex and most thoroughly described scenario in the requirements (section 6 of the requirements). Classes are distributed according to standard analysis stereotypes:

- «boundary» – user interaction elements: the folder watcher (FolderWatcherUI), the manual sync button (SyncNowButtonUI), the conflict resolution dialog (ConflictDialogUI);
- «control» – control logic: SyncController (client), SyncDiffService (server, diff computation), ConflictResolver (applying the user’s choice);
- «entity» – domain data: ManifestEntry, SyncCursor, File, ChangeLogEntry.

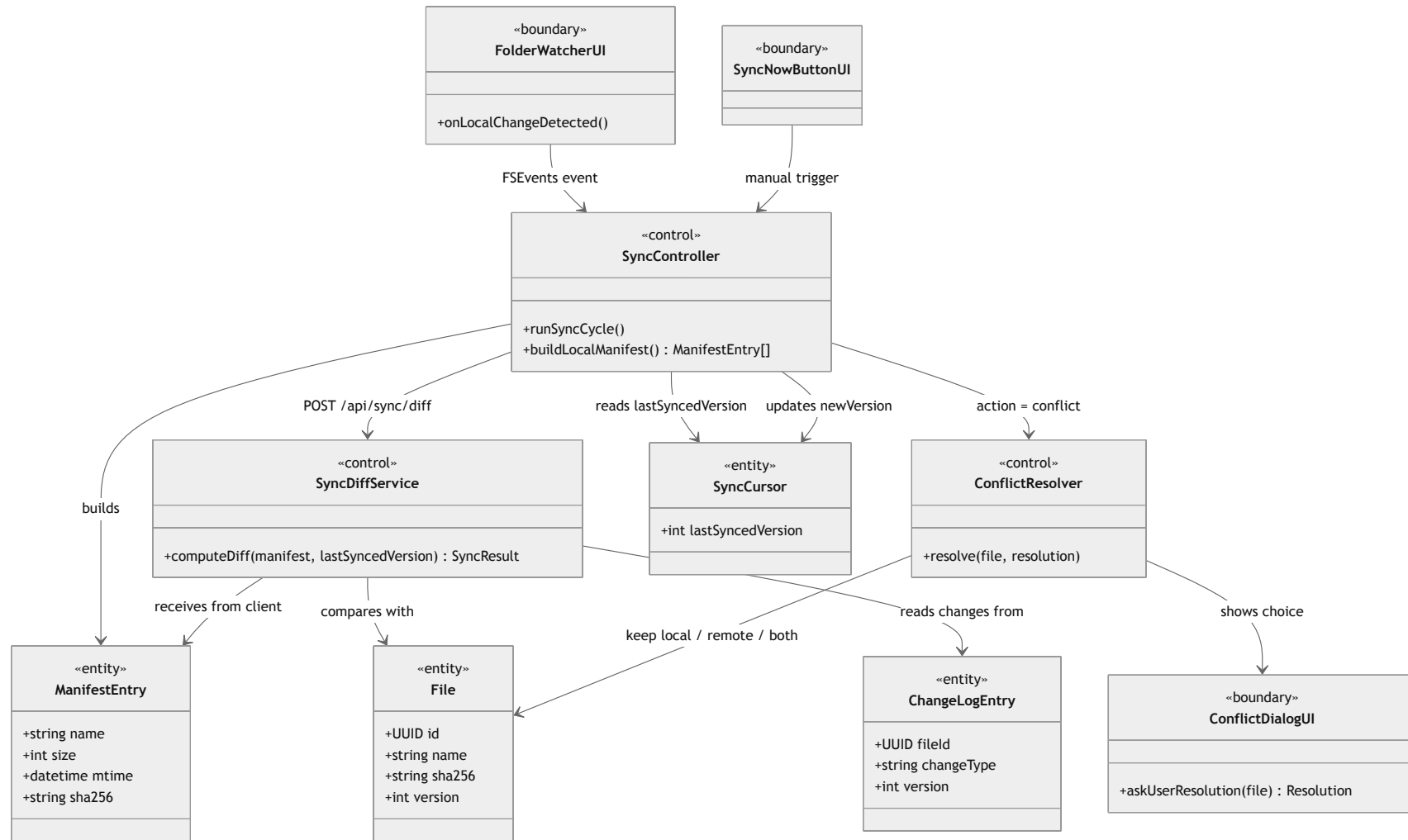


Figure 4: VOPC class diagram for the “Synchronize local folder with remote” use case

3.2 Object Diagram (Conflict Snapshot)

Unlike class diagrams, which describe the structure of types, an object diagram (fig. 5) captures a *concrete runtime snapshot* of instances – one moment of the synchronization scenario at which a conflict is detected (requirements section 6.2, the “exists, hash differs” case). The local instance `localFile` (version 44, computed on the client) and the server-side `remoteFile` (version 45) share the same name and `id`, but differ in `sha256` – the signature of edits made on both sides. The `cll : ChangeLogEntry` record confirms that version 45 is newer than the `lastSyncedVersion = 42` stored in the client’s cursor, i.e. the server-side change happened after the last successful sync.

Notation note: Mermaid has no built-in object diagram type, so the instances were built using `flowchart`: each object’s header is labeled per the UML convention as instance_name : Class, with concrete attribute values listed below a separator.

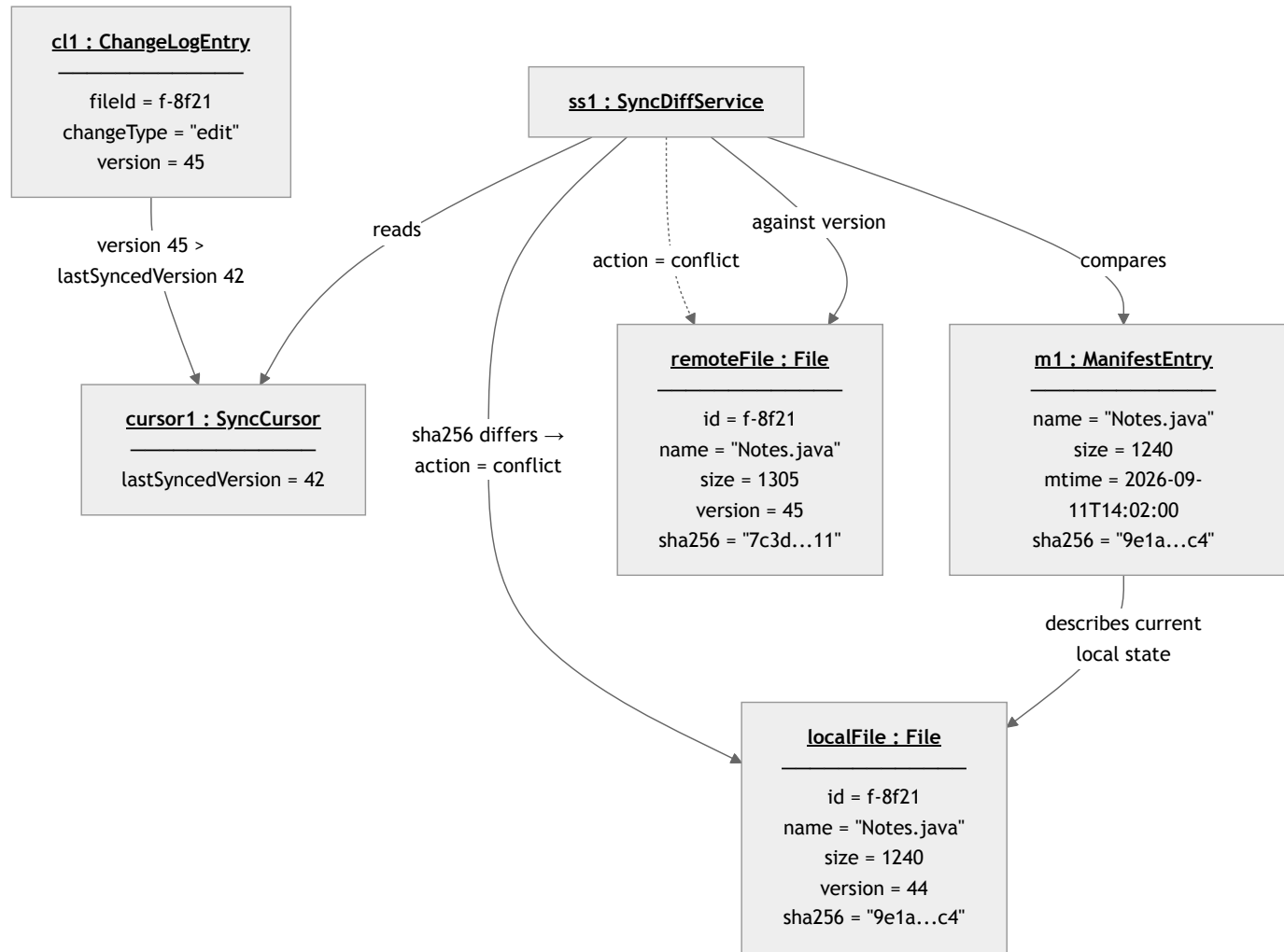


Figure 5: Object diagram: state snapshot during an editing conflict

3.3 Interaction Diagrams (Sequence & Communication Diagrams)

Three sequence diagrams are presented, illustrating key interaction scenarios between the client, the Go API server, and the database, together with a communication diagram as an alternative representation of one of these scenarios.

Authentication

Fig. 6 shows the sequence of verifying user credentials and issuing a token pair (access/refresh), consistent with the authentication description in the requirements (sections 1, 5.2).

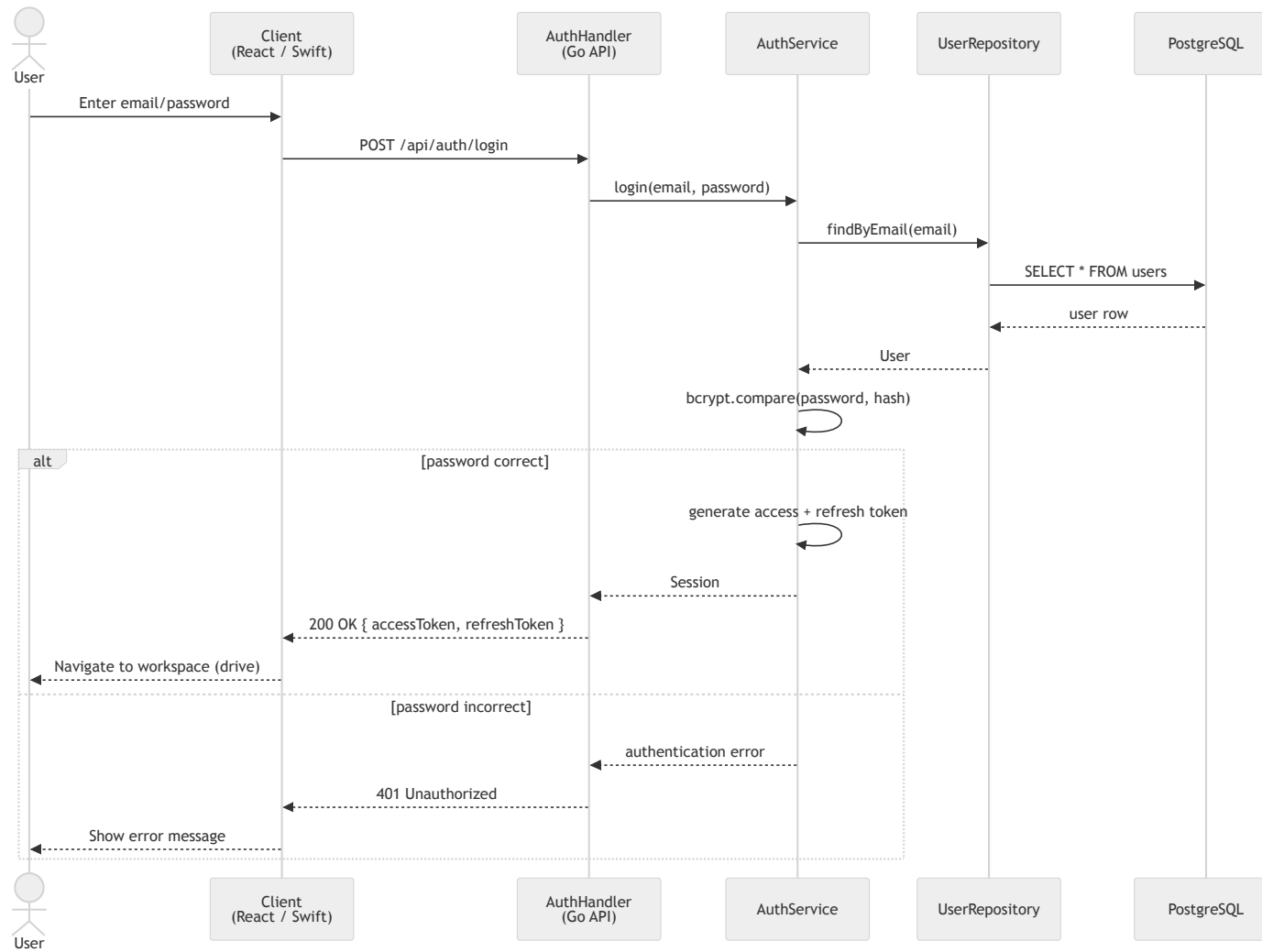


Figure 6: Sequence diagram: user authentication

File Upload

Fig. 7 illustrates the path of a request from the client through the handler layer to the service layer, which first writes the file content to storage (`FileStorage`), and after the write is confirmed, writes the file metadata to PostgreSQL via `FileRepository`.

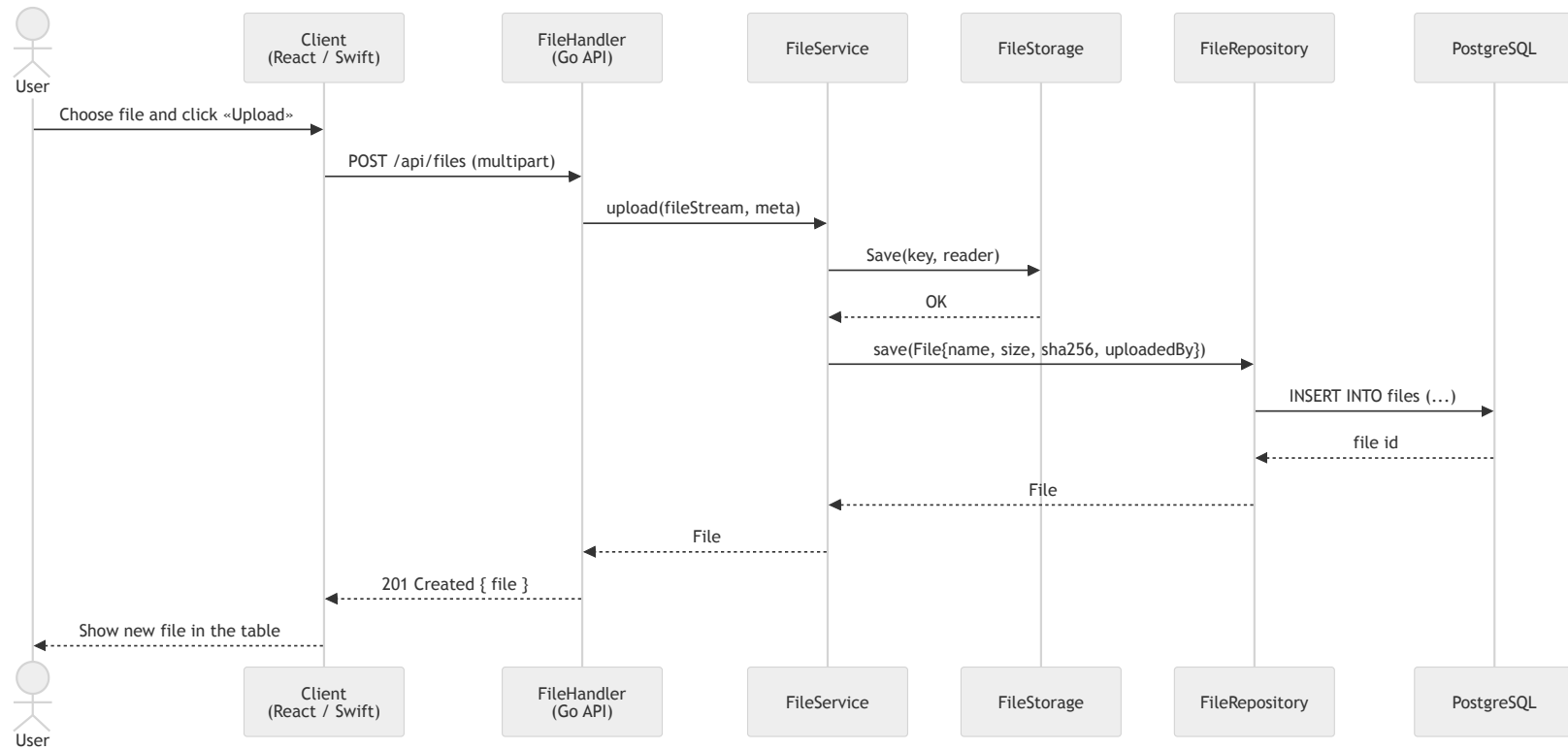


Figure 7: Sequence diagram: uploading a file to the server

Synchronization and Conflict Resolution

Fig. 8 details the `POST /api/sync/diff` protocol (section 6.6 of the requirements): the client sends the local manifest and `lastSyncedVersion`, the server returns a list of actions for each file, and conflicting files are handled through a request for the user's choice (keep local / keep remote / keep both).

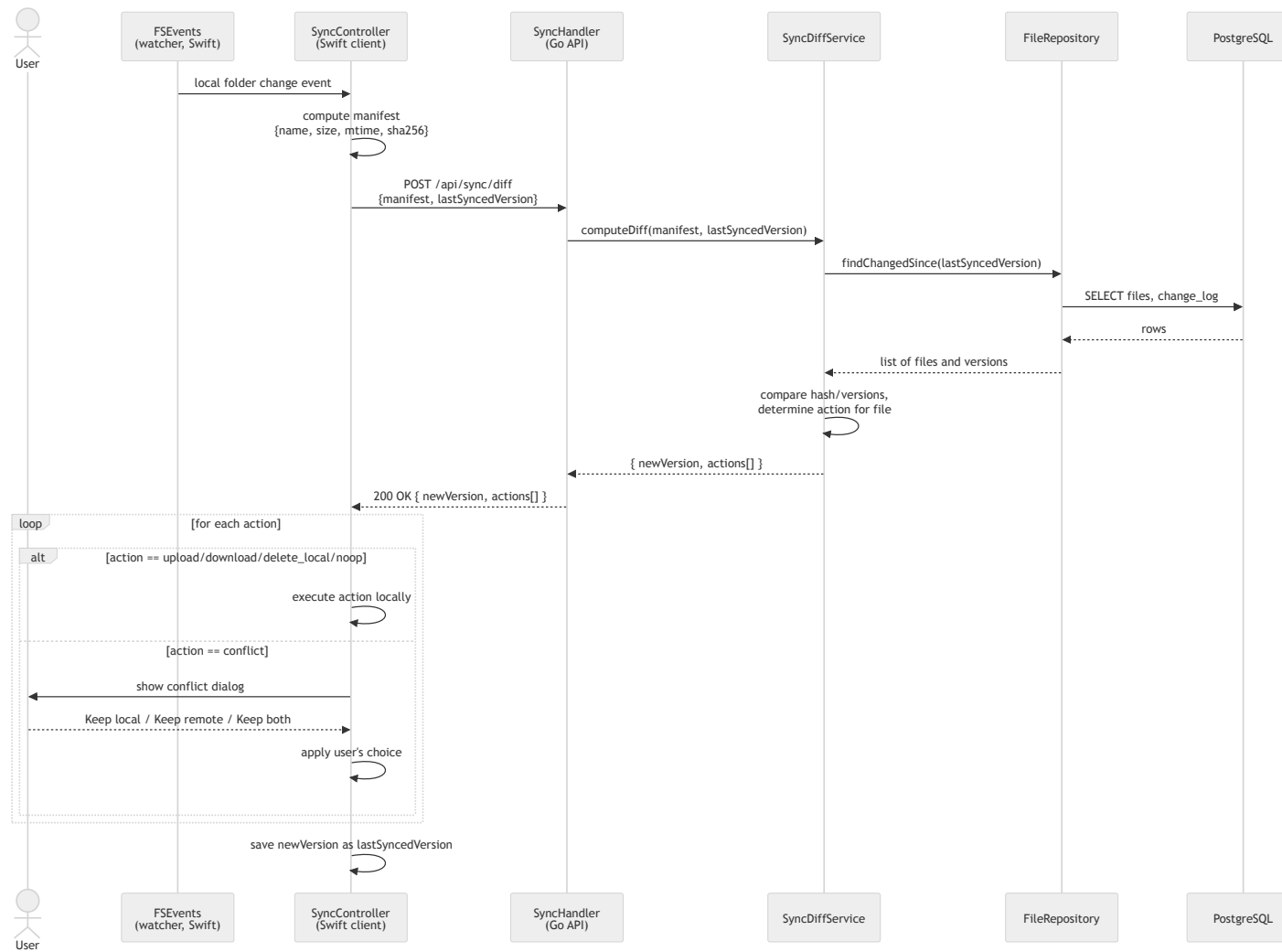


Figure 8: Sequence diagram: synchronization and conflict resolution

The Same Scenario as a Communication Diagram

Fig. 9 depicts **the same** synchronization and conflict-resolution scenario as fig. 8, but in communication-diagram notation: the emphasis shifts from the time axis to the static links between objects, and message ordering is conveyed through numbering (with decimal nesting for messages inside conditional branches – e.g. 5.1--5.3 for the user interaction in the conflict case). Both diagrams are consistent with each other – they use the same objects and messages – and together demonstrate the two standard ways of depicting one interaction.

Notation note: Mermaid has no built-in communication diagram type, so it was built using flowchart: objects are graph nodes, and the numbered edge labels correspond to messages.

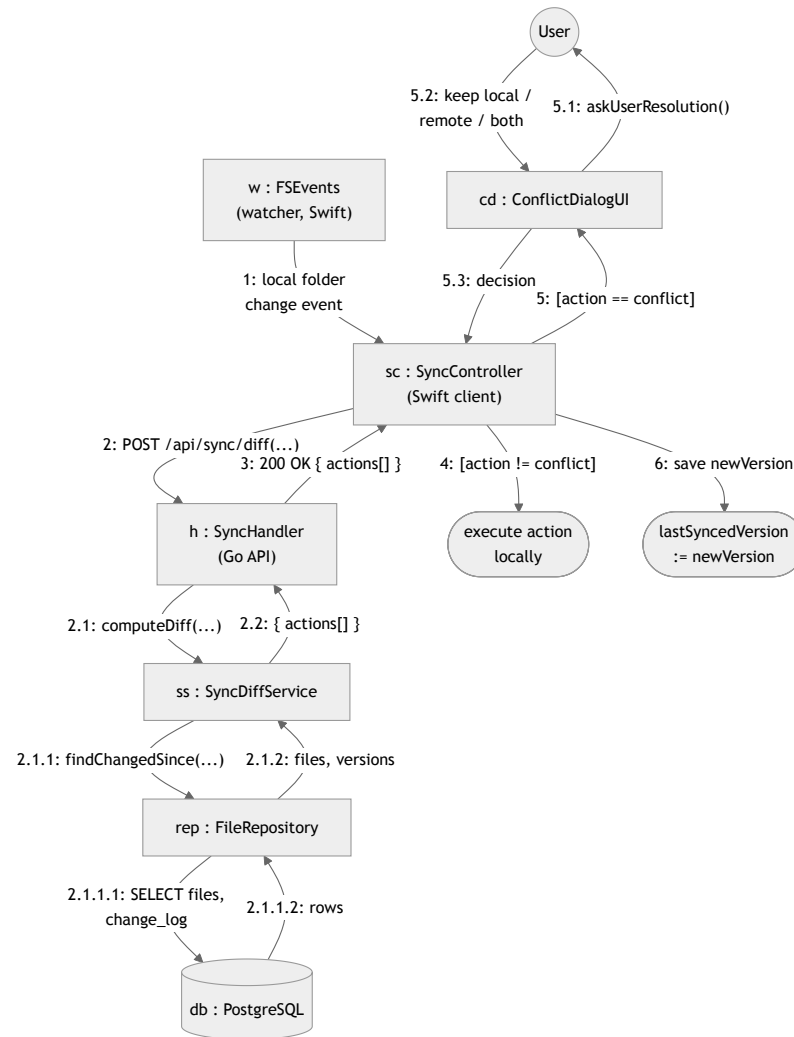


Figure 9: Communication diagram: synchronization and conflict resolution

3.4 State Diagram

Fig. 10 describes the behavior of the *File* entity depending on its synchronization state: from local/remote creation, through transfer (Uploading/Downloading), to the synchronized state, possible local or remote changes, and, in case of a simultaneous change on both sides, a conflict resolved by the user. The diagram directly implements the four manifest-comparison cases from section 6.2 of the requirements (upload / download / no-op / conflict) together with handling deletions via the version cursor (section 6.3).

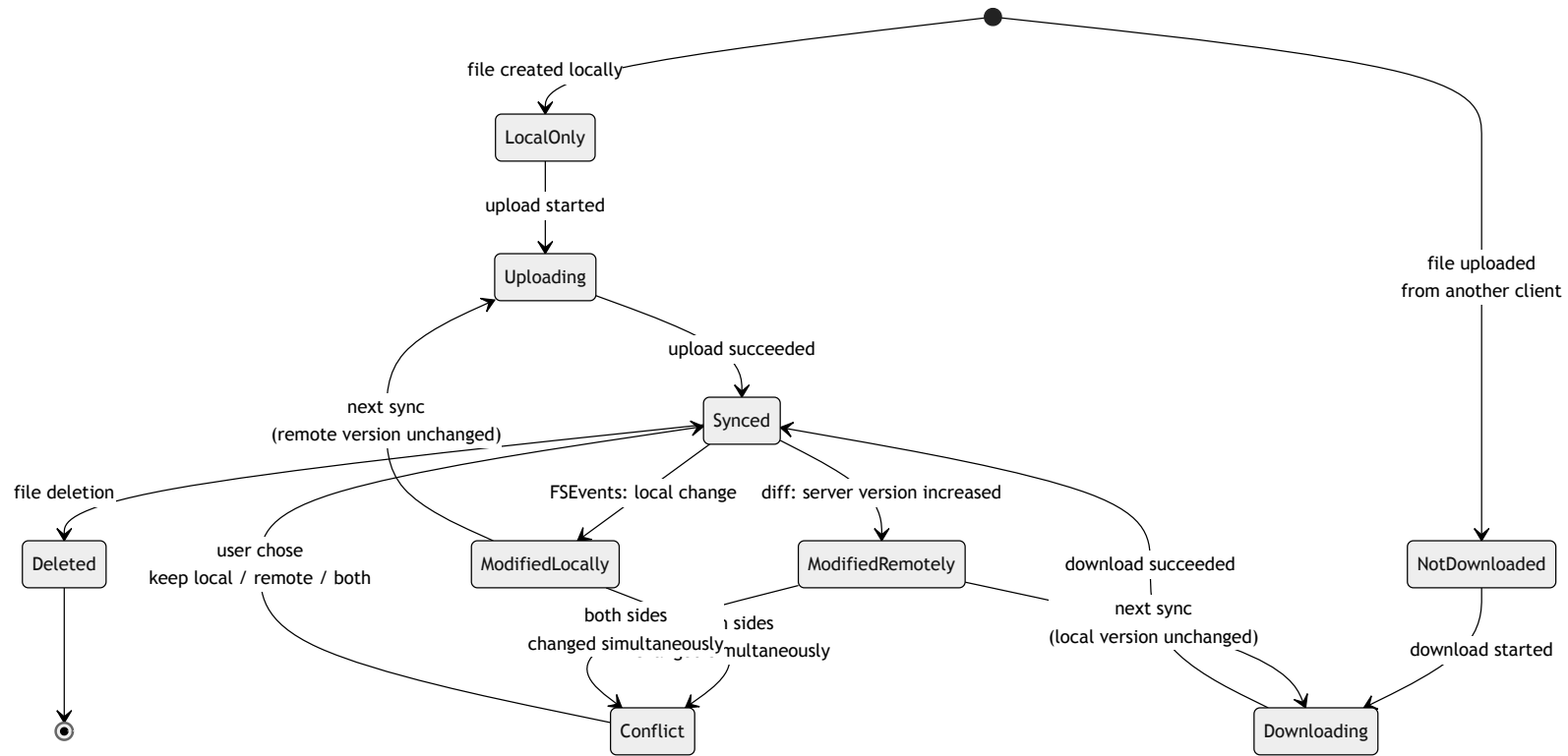


Figure 10: State diagram of a file during synchronization

3.5 Component Diagram

Fig. 11 details the software components of the system and their dependencies: the client applications, the cloudflared tunnel, and, inside the Go API server, the `Handler` → `Service` → `Repository` layers, together with the implementation-independent `FileStorage` interface, which allows swapping local storage for S3-compatible storage without changing the business logic (sections 5.2, 5.5 of the requirements).

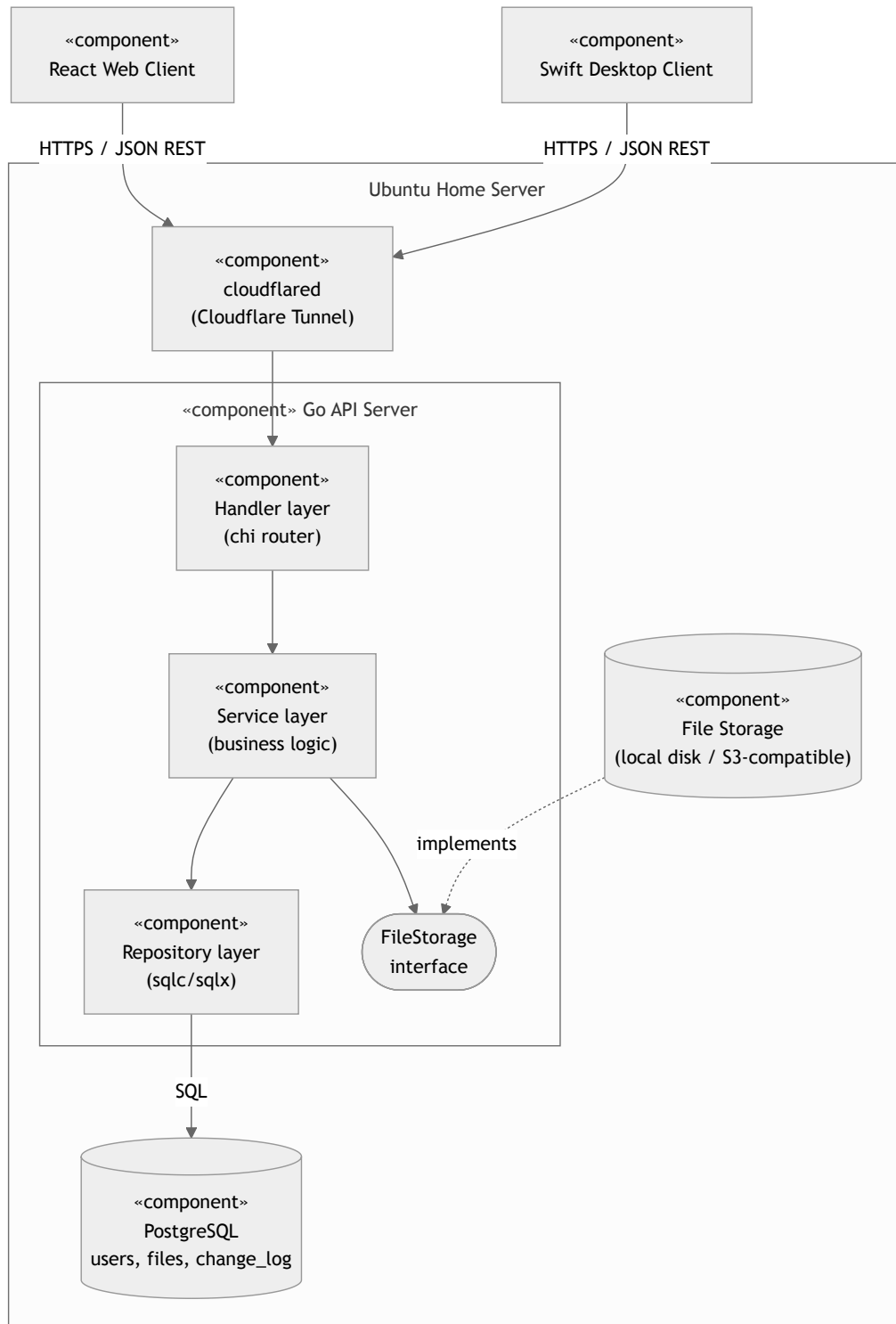


Figure 11: Component diagram of the system

3.6 Package Diagram

Where the component diagram (fig. 11) shows runtime components and the interfaces between them, the package diagram (fig. 12) shows the *static* organization of the code into modules, following the planned repository layout

(backend, frontend, desktop, migrations): inside the Go module, a layered package structure handler → service → repo/storage, matching the “no direct filesystem calls from handlers” convention and the principle of accessing storage only through the FileStorage interface. The repo package additionally depends on the schema defined in migrations.

Notation note: Mermaid has no built-in package diagram type, so packages are shown as nested subgraph blocks (with a «package» stereotype in the title), and dependencies as dashed arrows labeled with the dependency action.

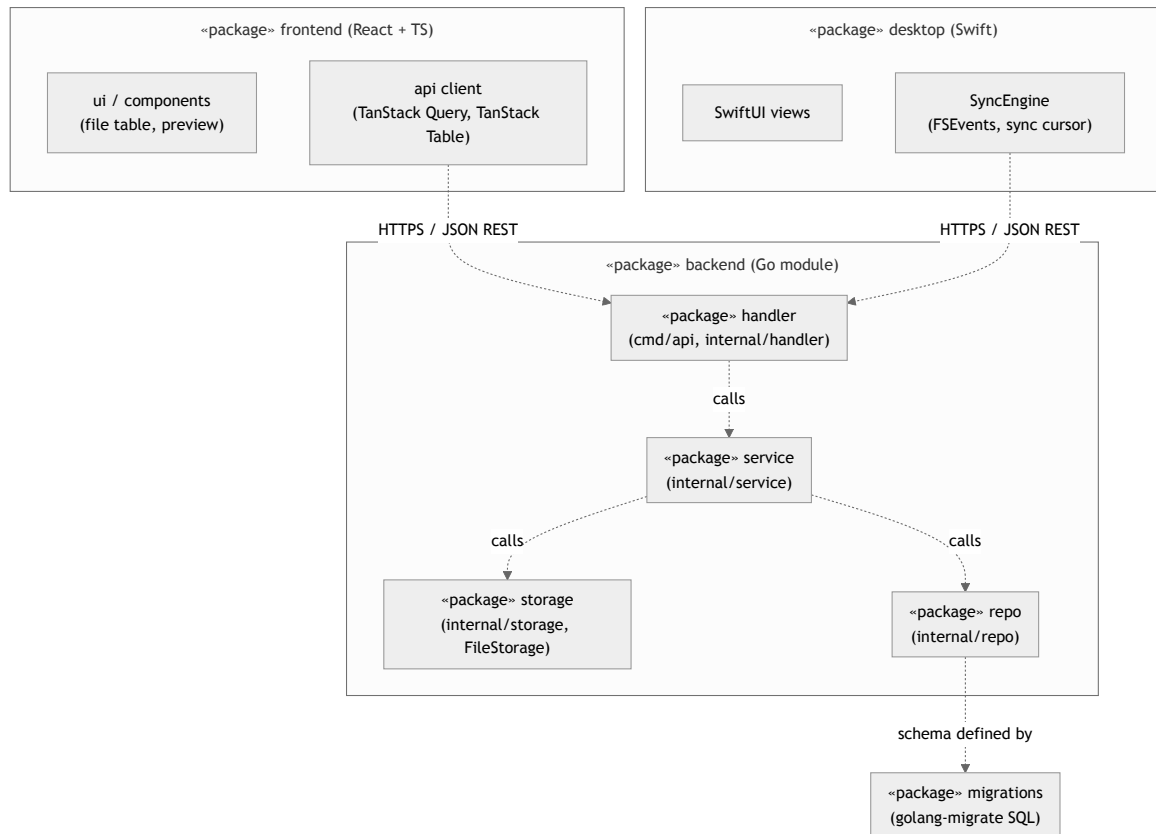


Figure 12: Package diagram: code organization by module

3.7 Activity Diagram

Fig. 13 details the business logic of a single synchronization cycle: computing the local manifest, sending it to the server, determining the action for each file (upload/download/no-op/conflict/delete_local), executing the action or, in the event of a conflict, obtaining the user’s decision (keep local / keep remote / keep both), and finally saving the new sync cursor version (section 6 of the requirements).

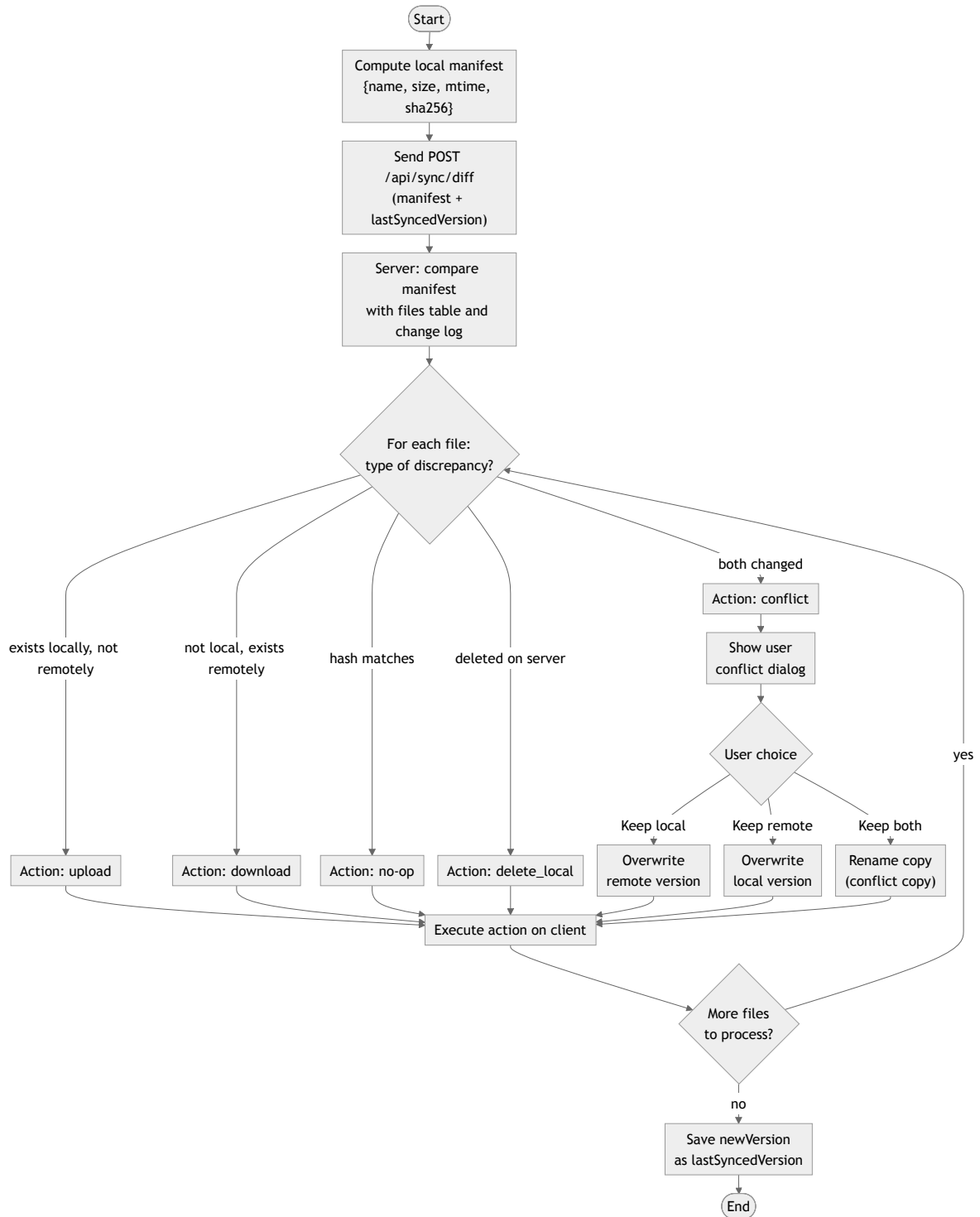


Figure 13: Activity diagram: synchronization cycle

4. Conclusions

The developed set of diagrams consistently reveals the Remote File Folder Client system from business requirements to detailed structure and behavior: the use case diagram captures the user's functional scenarios; the general

class diagram and the deployment diagram define the architectural framework; the VOPC diagram, the object diagram, the sequence and communication diagrams, and the state, component, package, and activity diagrams detail the implementation of the most complex scenario – synchronizing files between the local and remote folder, including the manifest-comparison protocol and editing-conflict resolution. In particular, the object diagram and the communication diagram complement the VOPC diagram and the sequence diagrams, respectively, with a concrete state snapshot and an alternative (structural rather than time-based) notation for the same scenario, while the package diagram captures the static modular organization of the code, consistent with the planned repository layout. The constructed diagrams are mutually consistent (shared classes, entities, and actions) and serve as a foundation for the further development and testing of the system.